LSTAT2150 Project - Template A

Your Name

2025-11-25

1 Exploratory analysis and kernel density estimation

We consider $n = 250$ observations (X_i, Y_i), $i = 1, \ldots, n$, drawn from a large population of households, where X_i represents the day of the year (measured as a continuous random variables between 0 and 365) and Y_i represents the total heating demand (in kWh/day) for that household on day X_i.

```
### Load your data here using:
# name <- "NOMA-template-A.RData"
# load(name)
# head(data)

### The second file, NOMA-template-A-test.RData, will be used in Question 12.
```

Question 1. Plot and interpret the scatterplot of the data and the histograms of X and Y. Do you see evidence of heteroskedasticity?

(Write your answer here.)

```
### An example of how an R-code chunk works.
### The out.width argument gives plots that take up 32% of the line width each.
### The fig.asp = 1 argument gives square plots.

# plot(data)
# hist(data$x, main = "", freq = FALSE, xlab = "x")
# hist(data$y, main = "", freq = FALSE, xlab = "y")
```

Question 2. We'd like to estimate the density of Y. Plot the Kernel Density Estimator (KDE) with Gaussian kernel and direct plug-in bandwidth. Around which values of Y are we most likely to see bias? And around which values of Y will the KDE be the most variable?

(Write your answer here.)

```
### Code to be written
```

Question 3. Compare (and visualize) the KDEs obtained using the normal reference bandwidth (also called Silverman's rule of thumb), the direct plug-in bandwidth, and least-squares cross-

validation (also called unbiased cross-validation). How do the different estimators perform close to the boundaries of the support of Y?

(Write your answer here.)

```
### Code to be written
```

Question 4. The code below can be used to generate bootstrap 95% confidence intervals for a KDE. Adapt the code to make a plot of the KDE based on the value of h you chose in Question 3 with 95% confidence intervals. Do the intervals become wider or narrower as h increases? Why? Would it be preferable to increase h?

(Write your answer here.)

```
### 95% bootstrap confidence intervals for some dataset Y
Y <- rnorm(200)
kde <- density(Y)

boot_y <- function(Y, lim){
  new_Y <- sample(Y, replace = TRUE)
  density(new_Y, from = lim[1], to = lim[2])$y
}

fits <- replicate(1000, boot_y(Y, range(kde$x)))
CIs <- t(apply(fits, 1, quantile, c(0.025,0.975)))

### Code to be written
```

Question 5. The code below generates a bivariate gaussian KDE. How can you interpret the plot? This estimator is based on a full bandwidth matrix H. Would it be a good idea here to set $H = hI_2$? Why (not)? *NB: you don't need to write any code for this question.*

(Write your answer here.)

```
library(ks)
# kde_biv <- kde(data)
# plot(kde_biv, display = "filled.contour", cont = c(25, 50, 75, 92.5, 99.75), lwd=1.5)
```

2 Non-parametric regression

The goal is to estimate $m(x) = \mathbb{E}[Y_i \mid X_i = x]$ using different methods and to compare the resulting estimates. You'll may want to use the `KernSmooth` and `splines` packages.

Question 6. The scatter plot suggests that a linear model, $m(x) = \beta_0 + \beta_1 x$, is not appropriate. What about a polynomial regression model, $m(x) = \beta_0 + \beta_1 x + \cdots + \beta_p x^p$? Fit this model for $p = 2$ and $p = 3$, plot the resulting estimates and comment on the results. Do you see evidence of heteroskedasticity (and is this in line with your answer to Question 1)?

(Write your answer here.)

```
### A third-degree polynomial regression
# fitpol3 <- lm(y ~ x + I(x^2) + I(x^3), data = data)

### Code to be written
```

Question 7. Plot the kernel regression (Nadaraya–Watson) estimator for three different values for h (one that looks OK visually, one that is undersmoothing and one that is oversmoothing). Would it be possible to use one of the bandwidths of Question 3 instead? Why (not)?

(Write your answer here.)

```
library(KernSmooth)

### Code to be written
```

Question 8. Fit a local linear estimator (i.e. a local polynomial estimator with $p = 1$) with a bandwidth h chosen as the direct plug-in bandwidth $\hat{h}_{\mathrm{rt}}$. In which regions do we expect the estimator to be the least biased? And where do we expect it to have the smallest variance? Compare the result with the Nadaraya–Watson estimate of the previous question (use the same h) and comment on the differences.

(Write your answer here.)

```
### Code to be written
```

Question 9. Fit and visualize the kNN regression estimator with tricube kernel and $p = 2$ for three different values of k (again, try to pick one that is visually convincing, one that's undersmoothing and one that's oversmoothing). Does it look like a good competitor of the local linear estimator above?

(Write your answer here.)

```
### you should apply the 'predict' function to
# loess(y ~ x, data = data)

### Code to be written
```

Question 10. Use cross-validation to pick an optimal bandwidth for the local linear estimator and an optimal number of neighbours k for the kNN estimator. For the local linear estimator, you may use the code provided in TP2 or the function `npregbw` in the package `np` (see example below). For the kNN estimator, you may use the `cvNN` function below.

Make a plot showing simultaneously (i) the polynomial regression estimator of Question 6 (ii) the local linear estimator with CV bandwidth and (iii) the kNN estimator with CV bandwidth. Can

you decide which one of the three estimators you'd prefer based on the plot? Do you think the CV criterion selected appropriate values for h and k?

(Write your answer here.)

```
cvNN <- function(X, Y, k){
  n <- length(X)
  CVknn <- numeric(length = n)
  for (i in 1:n) {
    train_data <- data.frame(X = X[-i], Y = Y[-i])
    Ypred <- predict(loess(Y ~ X, data = train_data, span = k/(n-1),
            control = loess.control(surface = "direct")), X[i])
    CVknn[i] <- (Y[i] - Ypred)^2
   }
  return(mean(CVknn))
}

library(np)
options(np.messages = FALSE)
# npregbw(y ~ x, data = data, regtype = "ll")

### Code to be written
```

Question 11. Fit a natural cubic spline estimator (you can use the `ns` function from the `splines` package) for (i) 4 equally-spaced interior knots and (ii) 4 knots chosen as equidistance quantiles of the covariate. Please comment on the fit. To how many degrees of freedom do these models correspond?

(Write your answer here.)

```
library(splines)

### Code to be written
```

Question 12. Fit a smoothing spline estimator with λ chosen by cross-validation (using the function below). Plot your grid of values of λ against CV(λ). What is the effective degrees of freedom of the model with this choice of λ? Doe the fit look better than that of the natural cubic spline?

(Write your answer here.)

```
cv_lambda <- function(x, y, lambda) {
  fit <- smooth.spline(x, y, lambda = lambda, all.knots = TRUE)
  yhat <- predict(fit, x)$y
  h <- fit$lev
  res <- (y - yhat) / (1 - h)
  return(mean(res^2))
}
```

```
### Code to be written
```

Question 13. Focus on at least **three** of the previously fitted models, i.e., pick at least three models among the

- polynomial regression of degree p (Question 6);
- local linear estimator with cross-validation or direct plug-in bandwidth (Question 8 and 10);
- kNN regression estimator with k chosen via cross-validation (Questions 9 and 10)
- natural cubic spline estimator with your choice and number of knots (Question 11)
- smoothing spline with λ chosen via cross-validation (Question 12).

Compare these models in terms of their **predictive performance** as follows. You've estimated the above models based on a dataset of size `n = 250`, the *training data*. The file `NOMA-template-A-test.RData` contains the *testing data*, i.e., `N = 50` additional observations that were not part of the training data. For each fitted regression model, you can now calculate predictions $\hat{y}_i$, associated with covariate observations x_i from the testing data, and calculate the **Mean Squared Prediction Error** (MSPE),

$$\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

Report the MSPE for the models you considered and conclude which model performs best according to this criterion. Is this in line with your previous conclusions?

(Write your answer here.)

```
### Load data_test here using:
# name <- "NOMA-template-A-test.RData"
# load(name)
# head(data_test)

### If you want to "predict" from a locpoly fit, you'll need to interpolate:
# fit <- locpoly(data$x, data$y, bandwidth = 15)
# f_pred <- approxfun(fit$x, fit$y)
# y_hat <- f_pred(data_test$x)

### Code to be written
```